Progressive disclosure means showing only the most important information first, then revealing more detail when the user asks for it (or when they prove they want it). In SendFlow it is the best way to keep the full power for experts while making the app usable for beginners.

Here is a practical way to implement it.

1. Three explicit complexity modes (recommended foundation)

Give the user a clear choice (or detect it from their first setup):

Mode	Who it is for	What is shown by default	What is hidden / behind "More"
Rider	Beginners & normal riders	Traffic lights + 1 recommendation + basic stats + map with key events	Almost everything else
Intermediate	Experienced riders	Rider view + velocity balance, bottoms details, simple histograms	Deep tables, raw numbers, advanced curves
Tuner / Expert	Mechanics & suspension tuners	Full current feature set	Nothing

- Store the choice in user settings (and allow changing it later).
- On first launch, ask a couple of simple questions (weight, experience level, "I just want simple feedback" vs "I want all the data") and default to Rider.
- The mode controls which components are rendered and which data is fetched/calculated (saves battery and cognitive load).

2. UI patterns that work well

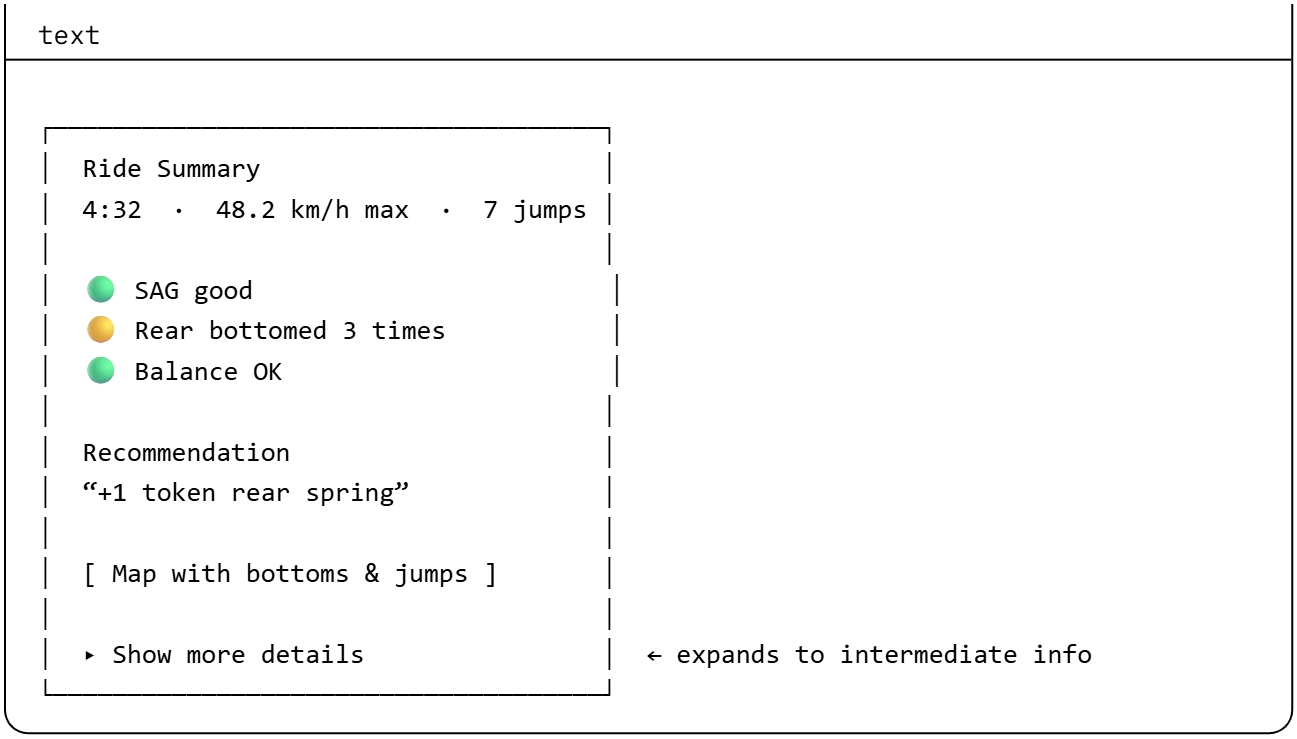
A. Summary → Details pattern (most important)

Every screen should follow this hierarchy:

1. **Hero summary** (always visible)
2. **Key metrics / traffic lights**
3. **"Show details" or expandable sections**
4. **Full advanced view** (only in Expert mode or after explicit expansion)

Example for the after-ride screen in Rider mode:

```
text
```



When the user taps “Show more details” you reveal:

- Simple front/rear travel used
- Rebound balance explanation
- List of the 3 biggest hits
- etc.

Only in Expert mode (or after another “Show advanced”) do you show histograms, velocity zones, full movement table, pressure curves, etc.

B. Expandable cards / accordions

Turn every dense section into a collapsible card:

- Closed state: title + one key number + traffic light
- Open state: the full graphs, tables, or explanations

This works especially well for:

- Velocity zones (HSR/LSR…)
- Event tables (bottoms, jumps, braking)
- Suspension group (temperature, pressure, fade)
- Taller / maintenance

C. Nested navigation or “Advanced” tab

Keep your current tabs, but:

- In Rider mode the tabs are simplified (Summary | Map | Setup | More)
- The “More” tab (or a long-press / overflow menu) leads to the full expert views
- Or add a persistent “Advanced” toggle at the top that switches the entire interface

D. Contextual “Why?” and “Details” buttons

Next to every traffic light or recommendation put a small “i” or “Why?”:

- First tap → plain-language explanation
- Second tap → the actual numbers and graphs that produced the recommendation

This educates users gradually without forcing complexity.

3. Implementation tips (technical)

State management

- Have a global `complexityMode` (or `userLevel`) in your app state / settings.
- Most screens should look something like:

```dart
// Pseudocode example (Flutter-style)
if (mode == Rider) {
  return RiderSummaryView(...);
} else if (mode == Intermediate) {
  return IntermediateView(...); // includes some expandable sections
} else {
  return ExpertFullView(...);   // everything you already have
}
```

Or more granular: each widget decides what to show based on the mode + whether the user has expanded that section.

Lazy loading / calculation

- In Rider mode, do not calculate or even load the heavy things (full histograms, velocity zone percentages, g-g diagrams, detailed event tables) until the user expands or switches mode.

- This keeps the app fast and the battery happier.

Persistence of expansions

- Remember which sections the user has expanded in the current session (or permanently if they want).
- Example: once someone opens the velocity zones three times, you can start showing a small preview by default.

Onboarding that teaches the layers

- First ride: only show the 4–5 traffic lights + recommendation.
- After a few rides, gently introduce “You can see more details here”.
- Never dump the full Expert interface on a new user.

4. Practical mapping to your current features

Feature group	Rider mode	Intermediate	Expert
PANEL (position, speeds)	Only SAG + bottoms + balance	+ average/max speeds	Full + zones + symmetry
LÍNEAS / time graph	Simple scrubber with key markers	+ basic stats	Full graph + all markers
DINÁMICA	Hidden or very simple	Simple histograms	Full + g-g + landings
EVENTOS (bottoms, jumps…)	Count + map markers + top 3	List with key numbers	Full table + all details
AJUSTES / Semáforos	Primary traffic lights	+ more detailed lights	Full engineer reading order
TALLER	Only “service due” warning	Basic history	Full history + degradation
TRACK GPS	Map with only important events	+ more layers	All layers + 3D profile
Video	Basic synced playback	+ overlays	Full clip export tools

5. Extra techniques that reinforce progressive disclosure

- **Smart defaults**: If the user has never looked at velocity zones, keep them collapsed forever until they open them once.

- **“Reset to simple” button**: Always available so someone who got curious and opened everything can go back to the clean Rider view.
- **Different home screens per mode**: The after-ride experience should feel completely different in Rider vs Expert.
- **Search / command palette** (advanced users love this): Let experts jump directly to any deep view while beginners never see the option.

Recommended starting point

1. Add the three-mode setting.
2. Redesign the after-ride / home screen so that in Rider mode it is only the traffic lights + one recommendation + map + basic stats.
3. Make every other dense section an expandable card.
4. Put the full current interface behind the Expert mode (or behind “Show advanced”).

This way you do not throw away any of the hard work you already did — you just control *when* and *to whom* it is shown.

Would you like a concrete wireframe description of the Rider after-ride screen, or example logic for how the traffic-light cards should expand into the underlying data?